In [1]:
```
from notebook_native_agent import start_agent, stop_agent, agent_status, agent_symbols, pause_agent, resume_agent, get_state

start_agent()
```

Notebook agent active

provider: openai | auto-execute: on | fallback: on | LLM cell classifier: off

## Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

### Try a natural-language cell like:

```
show me a funny fractal
```
copy

```
plot this: z = sin(x) * cos(y) (use plotly)
```
copy

```
I have this: tips.csv
```
copy

### How it works

1. **Stay in the notebook** — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.
2. **Work with live context** — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.
3. **Preview or auto-run** — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

**Fallback mode**

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

In [2]:
```
Create a fake coffee shop dataset and reveal the best-selling drinks with charts
```

## Notebook Native Agent

**What I understood**

Generate a synthetic coffee shop sales dataset and chart the best-selling drinks.

**Proposed solution**

Create a reusable fake sales dataset, then aggregate drink performance by **cups sold** and **revenue** to identify best sellers. I'll create notebook-friendly symbols including `coffee_shop_df` for the raw data, `drink_sales_df` for the drink-level summary, and `top_drinks_df` for the top performers, then display a preview, summary table, and bar charts.

**Code preview**

copy

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display


def generate_coffee_shop_data(n_orders: int = 1500, seed: int = 42) -> pd.DataFrame:
    """Generate a synthetic coffee shop order dataset."""
    rng = np.random.default_rng(seed)

    drinks = [
        "Latte", "Cappuccino", "Americano", "Mocha", "Espresso",
        "Cold Brew", "Flat White", "Chai Latte", "Matcha Latte", "Hot Chocolate"
    ]
    drink_weights = np.array([0.19, 0.15, 0.14, 0.10, 0.08, 0.11, 0.08, 0.06, 0.05, 0.04])
    base_prices = {
        "Latte": 4.50,
        "Cappuccino": 4.25,
        "Americano": 3.25,
        "Mocha": 4.95,
        "Espresso": 2.75,
        "Cold Brew": 4.10,
        "Flat White": 4.60,
        "Chai Latte": 4.80,
        "Matcha Latte": 5.10,
        "Hot Chocolate": 4.35,
    }
    size_options = ["Small", "Medium", "Large"]
    size_weights = np.array([0.25, 0.50, 0.25])
    size_price_add = {"Small": -0.30, "Medium": 0.0, "Large": 0.60}

    milk_options = ["Whole", "Oat", "Almond", "Soy", "Skim"]
    milk_weights = np.array([0.45, 0.22, 0.12, 0.09, 0.12])
    milk_price_add = {"Whole": 0.0, "Oat": 0.60, "Almond": 0.55, "Soy": 0.45, "Skim": 0.0}

    start_date = pd.Timestamp("2024-01-01")
    day_offsets = rng.integers(0, 90, size=n_orders)
    hour_choices = np.array([6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18])
    hour_weights = np.array([0.03, 0.07, 0.13, 0.15, 0.10, 0.08, 0.10, 0.08, 0.07, 0.07, 0.05, 0.04, 0.03])
    hours = rng.choice(hour_choices, size=n_orders, p=hour_weights)
    minutes = rng.integers(0, 60, size=n_orders)

    order_datetimes = start_date + pd.to_timedelta(day_offsets, unit="D") + pd.to_timedelta(hours, unit="h") + pd.to_timedelta(minutes, unit="m")
    selected_drinks = rng.choice(drinks, size=n_orders, p=drink_weights)
    selected_sizes = rng.choice(size_options, size=n_orders, p=size_weights)
    quantities = rng.choice([1, 2, 3], size=n_orders, p=[0.78, 0.18, 0.04])

    iced = []
    milk_type = []
    unit_price = []

    dairy_free_drinks = {"Americano", "Espresso", "Cold Brew"}
    hot_preferred_drinks = {"Espresso", "Cappuccino", "Flat White", "Hot Chocolate"}

    for drink, size in zip(selected_drinks, selected_sizes):
        if drink in dairy_free_drinks:
            chosen_milk = "None"
            milk_add = 0.0
        else:
            chosen_milk = rng.choice(milk_options, p=milk_weights)
            milk_add = milk_price_add[chosen_milk]
        milk_type.append(chosen_milk)

        if drink in hot_preferred_drinks:
            is_iced = rng.choice([False, True], p=[0.85, 0.15])
        elif drink in {"Cold Brew", "Matcha Latte", "Chai Latte"}:
            is_iced = rng.choice([False, True], p=[0.35, 0.65])
        else:
```

```
            is_iced = rng.choice([False, True], p=[0.60, 0.40])
        iced.append(is_iced)

        price = base_prices[drink] + size_price_add[size] + milk_add
        if is_iced:
            price += 0.20
        price += rng.normal(0, 0.08)
        unit_price.append(round(max(price, 2.25), 2))

    coffee_shop_df = pd.DataFrame({
        "order_id": np.arange(1, n_orders + 1),
        "order_datetime": order_datetimes,
        "drink": selected_drinks,
        "size": selected_sizes,
        "milk_type": milk_type,
        "iced": iced,
        "quantity": quantities,
        "unit_price": unit_price,
    })
    coffee_shop_df["revenue"] = (coffee_shop_df["quantity"] * coffee_shop_df["unit_price"]).round(2)
    coffee_shop_df["order_date"] = coffee_shop_df["order_datetime"].dt.date
    coffee_shop_df["hour"] = coffee_shop_df["order_datetime"].dt.hour

    return coffee_shop_df.sort_values("order_datetime").reset_index(drop=True)


def summarize_drink_sales(coffee_shop_df: pd.DataFrame) -> pd.DataFrame:
    """Aggregate drink-level sales metrics."""
    drink_sales_df = (
        coffee_shop_df.groupby("drink", as_index=False)
        .agg(
            orders=("order_id", "count"),
            cups_sold=("quantity", "sum"),
            total_revenue=("revenue", "sum"),
            avg_unit_price=("unit_price", "mean"),
        )
        .sort_values(["cups_sold", "total_revenue"], ascending=[False, False])
        .reset_index(drop=True)
    )
    drink_sales_df["total_revenue"] = drink_sales_df["total_revenue"].round(2)
    drink_sales_df["avg_unit_price"] = drink_sales_df["avg_unit_price"].round(2)
    return drink_sales_df


coffee_shop_df = generate_coffee_shop_data(n_orders=1800, seed=42)
drink_sales_df = summarize_drink_sales(coffee_shop_df)
top_drinks_df = drink_sales_df.head(10).copy()

print("Coffee shop dataset created:")
print(f"- Orders: {len(coffee_shop_df):,}")
print(f"- Date range: {coffee_shop_df['order_datetime'].min():%Y-%m-%d} to {coffee_shop_df['order_datetime'].max():%Y-%m-%d}")
print(f"- Best-selling drink by cups sold: {drink_sales_df.loc[0, 'drink']} ({int(drink_sales_df.loc[0, 'cups_sold']):} cups)")
print(f"- Top revenue drink: {drink_sales_df.sort_values('total_revenue', ascending=False).iloc[0]['drink']}")

display(coffee_shop_df.head(10))
display(drink_sales_df)

revenue_rank_df = drink_sales_df.sort_values("total_revenue", ascending=False).head(10).copy()

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].bar(top_drinks_df["drink"], top_drinks_df["cups_sold"], color="#6f4e37")
axes[0].set_title("Top Drinks by Cups Sold")
axes[0].set_xlabel("Drink")
axes[0].set_ylabel("Cups Sold")
axes[0].tick_params(axis="x", rotation=45)

axes[1].bar(revenue_rank_df["drink"], revenue_rank_df["total_revenue"], color="#c98c3a")
axes[1].set_title("Top Drinks by Revenue")
axes[1].set_xlabel("Drink")
axes[1].set_ylabel("Revenue ($)")
axes[1].tick_params(axis="x", rotation=45)

fig.tight_layout()
display(fig)
plt.close(fig)
```

**Execution**

auto-execute via gpt-5.4

**Selected context**

No prior notebook context looked necessary.

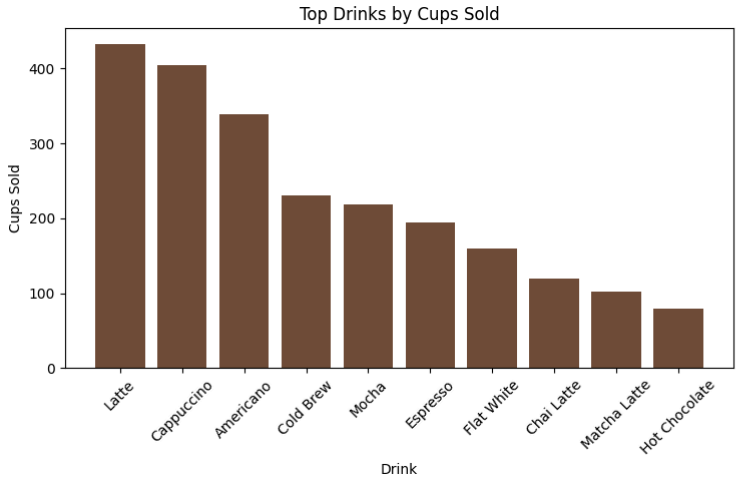
**Safety notes**

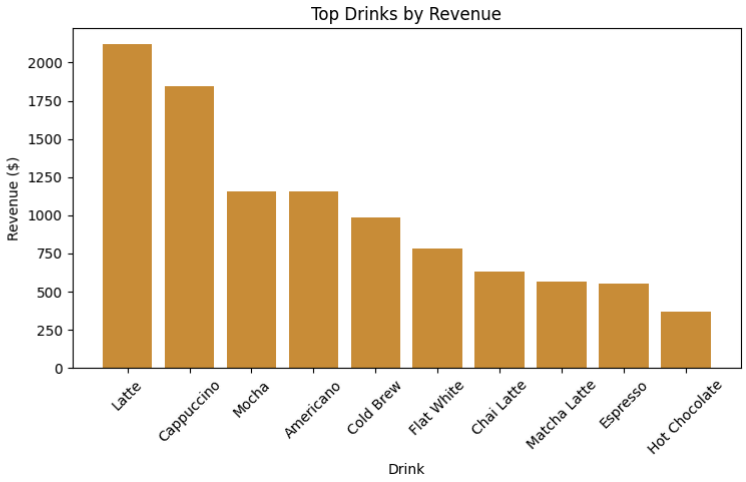
• Looks like an in-memory inspection or plotting task.

```
Coffee shop dataset created:
- Orders: 1,800
- Date range: 2024-01-01 to 2024-03-30
- Best-selling drink by cups sold: Latte (432 cups)
- Top revenue drink: Latte
```

| | order_id | order_datetime | drink | size | milk_type | iced | quantity | unit_price | revenue | order_date | hour |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1488 | 2024-01-01 07:21:00 | Americano | Medium | None | True | 1 | 3.54 | 3.54 | 2024-01-01 | 7 |
| **1** | 1294 | 2024-01-01 07:47:00 | Cappuccino | Large | Whole | False | 2 | 4.78 | 9.56 | 2024-01-01 | 7 |
| **2** | 104 | 2024-01-01 08:43:00 | Cold Brew | Medium | None | True | 1 | 4.42 | 4.42 | 2024-01-01 | 8 |
| **3** | 1237 | 2024-01-01 09:05:00 | Latte | Small | Whole | False | 1 | 4.24 | 4.24 | 2024-01-01 | 9 |
| **4** | 381 | 2024-01-01 09:11:00 | Latte | Large | Whole | True | 1 | 5.16 | 5.16 | 2024-01-01 | 9 |
| **5** | 1482 | 2024-01-01 09:15:00 | Espresso | Medium | None | False | 1 | 2.74 | 2.74 | 2024-01-01 | 9 |
| **6** | 1279 | 2024-01-01 09:54:00 | Americano | Small | None | True | 1 | 3.02 | 3.02 | 2024-01-01 | 9 |
| **7** | 1175 | 2024-01-01 10:08:00 | Latte | Medium | Soy | True | 1 | 5.16 | 5.16 | 2024-01-01 | 10 |
| **8** | 1190 | 2024-01-01 10:29:00 | Cold Brew | Medium | None | True | 1 | 4.36 | 4.36 | 2024-01-01 | 10 |
| **9** | 1763 | 2024-01-01 10:37:00 | Latte | Medium | Soy | True | 1 | 5.10 | 5.10 | 2024-01-01 | 10 |

| | drink | orders | cups_sold | total_revenue | avg_unit_price |
|---|---|---|---|---|---|
| **0** | Latte | 347 | 432 | 2118.22 | 4.90 |
| **1** | Cappuccino | 313 | 405 | 1848.58 | 4.56 |
| **2** | Americano | 259 | 339 | 1153.86 | 3.41 |
| **3** | Cold Brew | 183 | 231 | 986.48 | 4.30 |
| **4** | Mocha | 176 | 218 | 1157.80 | 5.32 |
| **5** | Espresso | 151 | 195 | 553.86 | 2.82 |
| **6** | Flat White | 124 | 159 | 783.48 | 4.93 |
| **7** | Chai Latte | 104 | 120 | 633.71 | 5.27 |
| **8** | Matcha Latte | 82 | 102 | 565.68 | 5.52 |
| **9** | Hot Chocolate | 61 | 79 | 370.26 | 4.73 |




In [3]: `Show me the best-selling drinks by time of day: morning, afternoon, evening.`

## Notebook Native Agent

**What I understood**

Analyze the existing coffee shop data and show the top-selling drinks for morning, afternoon, and evening.

**Proposed solution**

Reuse the existing `coffee_shop_df` and notebook imports (`pd`, `plt`, `display`). The code:

1. **Detects key columns** for drink name, timestamp, and quantity using common column-name patterns.
2. **Builds `time_of_day`** from the order hour when needed, using:
   - `morning`: 5:00–11:59
   - `afternoon`: 12:00–16:59
   - `evening`: 17:00–23:59
3. **Aggregates sales by drink and time of day** into `time_of_day_sales_df`.
4. Creates two reusable summaries:
   - `best_selling_by_time_df`: the top drink in each period
   - `top_drinks_by_time_df`: the top 5 drinks per period
5. **Displays** the summary tables and a small bar-chart panel for quick comparison.

If a quantity column is unavailable, it falls back to counting rows as one sale each.

**Code preview**

copy

```
def _find_first_column(df: pd.DataFrame, candidates: list[str]) -> str | None:
    lower_map = {str(col).lower(): col for col in df.columns}
    for candidate in candidates:
        if candidate.lower() in lower_map:
            return lower_map[candidate.lower()]
    for col in df.columns:
        col_lower = str(col).lower()
        if any(candidate.lower() in col_lower for candidate in candidates):
            return col
    return None


def _build_order_datetime(df: pd.DataFrame) -> tuple[pd.Series, str]:
    datetime_candidates = [
        "order_datetime", "transaction_datetime", "datetime", "timestamp",
        "order_time", "transaction_time", "sale_time", "created_at",
        "purchase_time", "date_time", "time"
    ]
    direct_col = _find_first_column(df, datetime_candidates)
    if direct_col is not None:
        order_dt = pd.to_datetime(df[direct_col], errors="coerce")
        if order_dt.notna().any() and order_dt.dt.hour.notna().any():
            return order_dt, str(direct_col)

    date_col = _find_first_column(df, ["date", "order_date", "transaction_date", "sale_date"])
    time_col = _find_first_column(df, ["time", "order_time", "transaction_time", "sale_time"])
    if date_col is not None and time_col is not None:
        combined = df[date_col].astype(str).str.strip() + " " + df[time_col].astype(str).str.strip()
        order_dt = pd.to_datetime(combined, errors="coerce")
        if order_dt.notna().any() and order_dt.dt.hour.notna().any():
            return order_dt, f"{date_col} + {time_col}"

    raise ValueError(
        "Could not identify a usable date/time column with hour information in coffee_shop_df."
    )


def _label_time_of_day(hour: float) -> str | None:
    if pd.isna(hour):
        return None
    hour = int(hour)
    if 5 <= hour < 12:
        return "morning"
    if 12 <= hour < 17:
        return "afternoon"
    if 17 <= hour < 24:
        return "evening"
    return None


def summarize_drinks_by_time_of_day(df: pd.DataFrame, top_n: int = 5) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict]:
    """Summarize top-selling drinks for morning, afternoon, and evening."""
    drink_col = _find_first_column(
        df,
        ["drink", "beverage", "item", "item_name", "product", "product_name", "menu_item"]
    )
    if drink_col is None:
```

```
        raise ValueError("Could not identify a drink/product column in coffee_shop_df.")

    quantity_col = _find_first_column(
        df,
        ["quantity", "qty", "cups_sold", "cups", "units", "count", "items_sold"]
    )

    order_dt, datetime_source = _build_order_datetime(df)

    analysis_df = df[[drink_col]].copy()
    analysis_df["order_dt"] = order_dt

    if quantity_col is not None:
        analysis_df["sales_units"] = pd.to_numeric(df[quantity_col], errors="coerce").fillna(0)
        sales_source = str(quantity_col)
    else:
        analysis_df["sales_units"] = 1
        sales_source = "row_count"

    analysis_df = analysis_df.dropna(subset=[drink_col, "order_dt"]).copy()
    analysis_df["time_of_day"] = analysis_df["order_dt"].dt.hour.map(_label_time_of_day)
    analysis_df = analysis_df.dropna(subset=["time_of_day"]).copy()

    time_order = ["morning", "afternoon", "evening"]
    analysis_df["time_of_day"] = pd.Categorical(analysis_df["time_of_day"], categories=time_order, ordered=True)

    time_of_day_sales_df = (
        analysis_df
        .groupby(["time_of_day", drink_col], observed=True, as_index=False)["sales_units"]
        .sum()
        .sort_values(["time_of_day", "sales_units", drink_col], ascending=[True, False, True])
        .reset_index(drop=True)
        .rename(columns={drink_col: "drink"})
    )

    best_selling_by_time_df = (
        time_of_day_sales_df
        .groupby("time_of_day", observed=True, as_index=False)
        .head(1)
        .reset_index(drop=True)
    )

    top_drinks_by_time_df = (
        time_of_day_sales_df
        .groupby("time_of_day", observed=True, group_keys=False)
        .head(top_n)
        .reset_index(drop=True)
    )

    metadata = {
        "drink_col": str(drink_col),
        "quantity_col": str(quantity_col) if quantity_col is not None else None,
        "datetime_source": datetime_source,
        "sales_source": sales_source,
    }
    return analysis_df, time_of_day_sales_df, best_selling_by_time_df, top_drinks_by_time_df, metadata


analysis_df, time_of_day_sales_df, best_selling_by_time_df, top_drinks_by_time_df, time_of_day_metadata = summarize_drinks_by_time_of_day(coffee_shop_df, top_n=5)

print("Best-selling drinks by time of day")
print(f"Using drink column: {time_of_day_metadata['drink_col']}")
print(f"Using datetime source: {time_of_day_metadata['datetime_source']}")
print(f"Using sales measure: {time_of_day_metadata['sales_source']}")

display(best_selling_by_time_df)
display(top_drinks_by_time_df)

periods = ["morning", "afternoon", "evening"]
fig, axes = plt.subplots(1, 3, figsize=(16, 4), constrained_layout=True)

for ax, period in zip(axes, periods):
    period_df = top_drinks_by_time_df[top_drinks_by_time_df["time_of_day"] == period].copy()
    if period_df.empty:
        ax.set_title(period.title())
        ax.text(0.5, 0.5, "No data", ha="center", va="center")
        ax.axis("off")
        continue

    period_df = period_df.sort_values("sales_units", ascending=True)
    ax.barh(period_df["drink"], period_df["sales_units"], color="#8c564b")
    ax.set_title(period.title())
    ax.set_xlabel("Units Sold")
```

```
fig.suptitle("Top 5 Drinks by Time of Day", y=1.05)
display(fig)
plt.close(fig)
```

**Execution**

auto-execute via gpt-5.4

**Selected context**

- `exec_1` ran generated code to create a fake coffee shop dataset and display best-selling drinks with charts.
- The code imported `numpy`, `pandas`, `matplotlib.pyplot`, and `display` from `IPython.display`.
- A `display` function is available in the notebook context.
- `axes` is an ndarray of Matplotlib axes, including charts titled **Top Drinks by Cups Sold** and **Top Drinks by Revenue**.

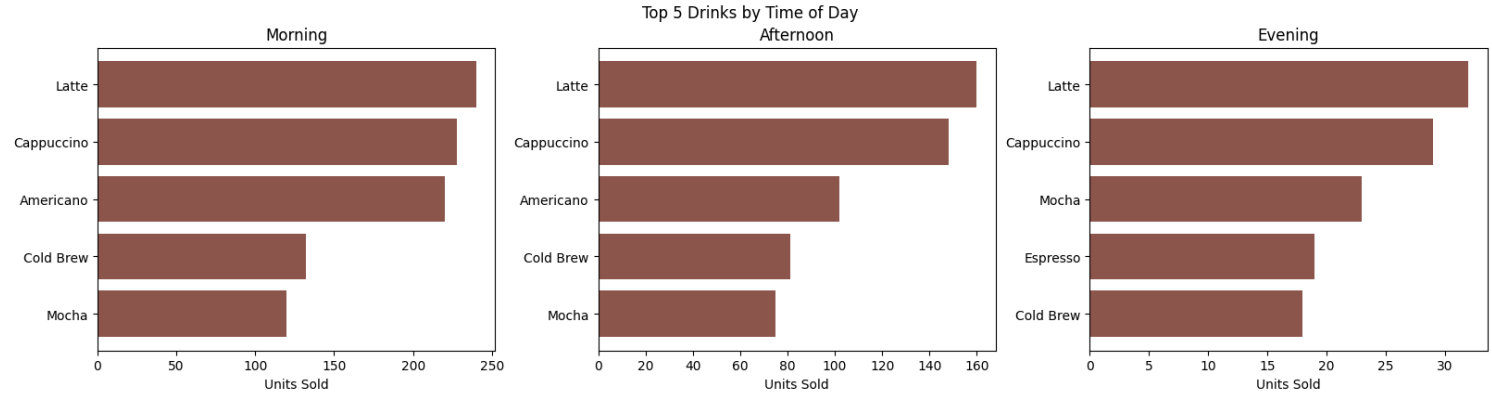
**Safety notes**

• Looks like an in-memory inspection or plotting task.

```
Best-selling drinks by time of day
Using drink column: drink
Using datetime source: order_datetime
Using sales measure: quantity
```

| | time_of_day | drink | sales_units |
|---|---|---|---|
| **0** | morning | Latte | 240 |
| **1** | afternoon | Latte | 160 |
| **2** | evening | Latte | 32 |

| | time_of_day | drink | sales_units |
|---|---|---|---|
| **0** | morning | Latte | 240 |
| **1** | morning | Cappuccino | 228 |
| **2** | morning | Americano | 220 |
| **3** | morning | Cold Brew | 132 |
| **4** | morning | Mocha | 120 |
| **5** | afternoon | Latte | 160 |
| **6** | afternoon | Cappuccino | 148 |
| **7** | afternoon | Americano | 102 |
| **8** | afternoon | Cold Brew | 81 |
| **9** | afternoon | Mocha | 75 |
| **10** | evening | Latte | 32 |
| **11** | evening | Cappuccino | 29 |
| **12** | evening | Mocha | 23 |
| **13** | evening | Espresso | 19 |
| **14** | evening | Cold Brew | 18 |




In [5]: `Create a mini business dashboard: KPIs, top drinks, revenue trend, category mix.`

## Notebook Native Agent

**What I understood**

Build a compact coffee shop business dashboard with KPIs, top drinks, a revenue trend, and category mix.

**Proposed solution**

I'll reuse an existing notebook dataframe when available ( `coffee_shop_df` , then `analysis_df` , then `source_df` ) and fall back to `generate_coffee_shop_data()` only if needed. The code also reuses `infer_column` when present to detect likely date, product, category, quantity, revenue, and order columns.

**Approach**

1. Select the best available sales dataframe.
2. Infer important columns and create safe fallbacks where needed.
3. Build reusable outputs:
- `dashboard_df`
- `kpi_df`
- `top_drinks_df`
- `revenue_trend_df`
- `category_mix_df`
- `dashboard_metadata`
4. Render a 2x2 matplotlib dashboard with:
- KPI cards
- top drinks bar chart
- revenue trend line chart
- category mix pie/bar chart

**Key reuse decisions**

- Reuses existing dataset symbols instead of regenerating data unnecessarily.
- Reuses `infer_column` if already defined.
- Keeps derived tables in named symbols for likely follow-up analysis.

**Code preview**

copy

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from IPython.display import display


def _fallback_infer_column(df: pd.DataFrame, candidates: list[str], allow_object_fallback: bool = False):
    lower_map = {str(col).lower(): col for col in df.columns}

    for candidate in candidates:
        if candidate.lower() in lower_map:
            return lower_map[candidate.lower()]

    for candidate in candidates:
        candidate_lower = candidate.lower()
        for lower_name, original_name in lower_map.items():
            if candidate_lower in lower_name or lower_name in candidate_lower:
                return original_name

    if allow_object_fallback:
        object_cols = [col for col in df.columns if df[col].dtype == "object"]
        if object_cols:
            return object_cols[0]

    return None


def _infer_col(df: pd.DataFrame, candidates: list[str], allow_object_fallback: bool = False):
    infer_fn = globals().get("infer_column")
    if callable(infer_fn):
        return infer_fn(df, candidates, allow_object_fallback=allow_object_fallback)
    return _fallback_infer_column(df, candidates, allow_object_fallback=allow_object_fallback)


def _pick_dashboard_df() -> tuple[pd.DataFrame, str]:
    for name in ["coffee_shop_df", "analysis_df", "source_df"]:
        obj = globals().get(name)
        if isinstance(obj, pd.DataFrame) and not obj.empty:
            return obj, name

    generator = globals().get("generate_coffee_shop_data")
    if callable(generator):
        generated_df = generator()
        globals()["coffee_shop_df"] = generated_df
        return generated_df, "coffee_shop_df"
```

```
    raise ValueError("No usable dataframe found. Expected one of coffee_shop_df, analysis_df, source_df, or generate_coffee_shop_data().")


def _format_currency(value: float) -> str:
    return f"${value:,.0f}"


def _format_number(value: float) -> str:
    if pd.isna(value):
        return "N/A"
    if float(value).is_integer():
        return f"{int(value):,}"
    return f"{value:,.2f}"


def build_business_dashboard_artifacts(df: pd.DataFrame) -> dict:
    """Prepare reusable summary tables for a compact business dashboard."""
    dashboard_df = df.copy()

    date_col = _infer_col(dashboard_df, ["order_datetime", "datetime", "timestamp", "date", "order_date", "transaction_date", "sale_date"])
    product_col = _infer_col(dashboard_df, ["drink", "product", "item", "menu_item", "beverage", "name"], allow_object_fallback=True)
    category_col = _infer_col(dashboard_df, ["category", "product_category", "drink_category", "type", "segment"])
    quantity_col = _infer_col(dashboard_df, ["quantity", "qty", "units", "count"])
    revenue_col = _infer_col(dashboard_df, ["revenue", "sales", "total_revenue", "amount", "order_total", "line_total", "total"])
    unit_price_col = _infer_col(dashboard_df, ["unit_price", "price", "avg_price"])
    order_col = _infer_col(dashboard_df, ["order_id", "transaction_id", "receipt_id", "ticket_id", "order"])

    if product_col is None:
        raise ValueError("Could not infer a product column for the dashboard.")

    if date_col is not None:
        dashboard_df[date_col] = pd.to_datetime(dashboard_df[date_col], errors="coerce")

    if quantity_col is None:
        quantity_col = "_dashboard_quantity"
        dashboard_df[quantity_col] = 1
    else:
        dashboard_df[quantity_col] = pd.to_numeric(dashboard_df[quantity_col], errors="coerce").fillna(0)

    if revenue_col is None and unit_price_col is not None:
        dashboard_df[unit_price_col] = pd.to_numeric(dashboard_df[unit_price_col], errors="coerce")
        revenue_col = "_dashboard_revenue"
        dashboard_df[revenue_col] = dashboard_df[quantity_col] * dashboard_df[unit_price_col]
    elif revenue_col is not None:
        dashboard_df[revenue_col] = pd.to_numeric(dashboard_df[revenue_col], errors="coerce")

    if category_col is None:
        category_col = "_dashboard_category"
        dashboard_df[category_col] = "Uncategorized"

    total_revenue = float(dashboard_df[revenue_col].fillna(0).sum()) if revenue_col is not None else np.nan
    total_units = float(dashboard_df[quantity_col].fillna(0).sum())
    total_orders = int(dashboard_df[order_col].nunique()) if order_col is not None else int(len(dashboard_df))
    avg_order_value = (total_revenue / total_orders) if revenue_col is not None and total_orders else np.nan

    kpi_df = pd.DataFrame(
        {
            "metric": ["Total Revenue", "Orders", "Units Sold", "Avg Order Value"],
            "raw_value": [total_revenue, total_orders, total_units, avg_order_value],
            "display_value": [
                _format_currency(total_revenue) if revenue_col is not None else "N/A",
                _format_number(total_orders),
                _format_number(total_units),
                _format_currency(avg_order_value) if revenue_col is not None and not pd.isna(avg_order_value) else "N/A",
            ],
        }
    )

    top_metric_name = "units_sold"
    top_drinks_df = (
        dashboard_df.groupby(product_col, dropna=False)[quantity_col]
        .sum()
        .sort_values(ascending=False)
        .reset_index()
        .rename(columns={product_col: "product", quantity_col: top_metric_name})
    )

    if date_col is not None and dashboard_df[date_col].notna().any():
        dated_df = dashboard_df.dropna(subset=[date_col]).copy()
        span_days = max((dated_df[date_col].max() - dated_df[date_col].min()).days, 0)

        if span_days > 120:
            trend_freq = "M"
```

```
        trend_label = "Month"
    elif span_days > 21:
        trend_freq = "W"
        trend_label = "Week"
    else:
        trend_freq = "D"
        trend_label = "Day"

    dated_df["trend_period"] = dated_df[date_col].dt.to_period(trend_freq).dt.to_timestamp()
    trend_value_col = revenue_col if revenue_col is not None else quantity_col
    trend_value_name = "revenue" if revenue_col is not None else "volume"

    revenue_trend_df = (
        dated_df.groupby("trend_period", as_index=False)[trend_value_col]
        .sum()
        .rename(columns={"trend_period": "period", trend_value_col: trend_value_name})
        .sort_values("period")
    )
else:
    trend_label = "Unavailable"
    trend_value_name = "revenue" if revenue_col is not None else "volume"
    revenue_trend_df = pd.DataFrame(columns=["period", trend_value_name])

category_value_col = revenue_col if revenue_col is not None else quantity_col
category_value_name = "revenue" if revenue_col is not None else "units_sold"
category_mix_df = (
    dashboard_df.groupby(category_col, dropna=False)[category_value_col]
    .sum()
    .sort_values(ascending=False)
    .reset_index()
    .rename(columns={category_col: "category", category_value_col: category_value_name})
)

dashboard_metadata = {
    "date_col": date_col,
    "product_col": product_col,
    "category_col": category_col,
    "quantity_col": quantity_col,
    "revenue_col": revenue_col,
    "order_col": order_col,
    "trend_period_label": trend_label,
    "trend_value_name": trend_value_name,
    "category_value_name": category_value_name,
}

return {
    "dashboard_df": dashboard_df,
    "kpi_df": kpi_df,
    "top_drinks_df": top_drinks_df,
    "revenue_trend_df": revenue_trend_df,
    "category_mix_df": category_mix_df,
    "dashboard_metadata": dashboard_metadata,
}


def plot_business_dashboard(kpi_df: pd.DataFrame, top_drinks_df: pd.DataFrame, revenue_trend_df: pd.DataFrame, category_mix_df: pd.DataFrame, dashboard_metadata: dict,
source_name: str):
    """Render a compact 2x2 business dashboard figure."""
    fig, axes = plt.subplots(2, 2, figsize=(15, 10), constrained_layout=True)
    fig.suptitle(f"Coffee Shop Mini Dashboard ({source_name})", fontsize=16, fontweight="bold")

    ax_kpi = axes[0, 0]
    ax_kpi.axis("off")
    ax_kpi.set_title("KPIs", loc="left", fontsize=13, fontweight="bold")

    positions = [(0.02, 0.55), (0.52, 0.55), (0.02, 0.08), (0.52, 0.08)]
    box_facecolors = ["#E8F1FB", "#F6E8FB", "#EAF7EA", "#FFF4D6"]

    for (x, y), (_, row), facecolor in zip(positions, kpi_df.iterrows(), box_facecolors):
        ax_kpi.text(
            x,
            y,
            f"{row['metric']}\n{row['display_value']}",
            transform=ax_kpi.transAxes,
            fontsize=13,
            fontweight="bold",
            va="top",
            ha="left",
            bbox=dict(boxstyle="round,pad=0.6", facecolor=facecolor, edgecolor="#BBBBBB"),
        )

    ax_top = axes[0, 1]
    top_n = min(8, len(top_drinks_df))
    top_plot_df = top_drinks_df.head(top_n).sort_values("units_sold", ascending=True)
```

```
    ax_top.barh(top_plot_df["product"].astype(str), top_plot_df["units_sold"], color="#8D6E63")
    ax_top.set_title("Top Drinks", loc="left", fontsize=13, fontweight="bold")
    ax_top.set_xlabel("Units Sold")
    ax_top.set_ylabel("")

    ax_trend = axes[1, 0]
    trend_value_name = dashboard_metadata["trend_value_name"]
    if not revenue_trend_df.empty:
        ax_trend.plot(revenue_trend_df["period"], revenue_trend_df[trend_value_name], marker="o", color="#2E86AB", linewidth=2)
        ax_trend.set_title(f"{trend_value_name.replace('_', ' ').title()} Trend by {dashboard_metadata['trend_period_label']}", loc="left", fontsize=13, fontweight="bold")
        ax_trend.set_xlabel("")
        ax_trend.tick_params(axis="x", rotation=30)
        if trend_value_name == "revenue":
            ax_trend.yaxis.set_major_formatter(FuncFormatter(lambda x, pos: f"${x:,.0f}"))
        ax_trend.grid(alpha=0.25)
    else:
        ax_trend.axis("off")
        ax_trend.text(0.05, 0.5, "No valid date column available for trend analysis.", fontsize=12, transform=ax_trend.transAxes)

    ax_mix = axes[1, 1]
    category_value_name = dashboard_metadata["category_value_name"]
    ax_mix.set_title("Category Mix", loc="left", fontsize=13, fontweight="bold")
    if len(category_mix_df) <= 1:
        ax_mix.bar(category_mix_df["category"].astype(str), category_mix_df[category_value_name], color="#7CB342")
        if category_value_name == "revenue":
            ax_mix.yaxis.set_major_formatter(FuncFormatter(lambda x, pos: f"${x:,.0f}"))
        ax_mix.tick_params(axis="x", rotation=20)
    else:
        pie_values = category_mix_df[category_value_name].fillna(0)
        pie_labels = category_mix_df["category"].astype(str)
        ax_mix.pie(pie_values, labels=pie_labels, autopct="%1.1f%%", startangle=90, counterclock=False)
        ax_mix.axis("equal")

    return fig


selected_df, dashboard_source_name = _pick_dashboard_df()
dashboard_artifacts = build_business_dashboard_artifacts(selected_df)

dashboard_df = dashboard_artifacts["dashboard_df"]
kpi_df = dashboard_artifacts["kpi_df"]
top_drinks_df = dashboard_artifacts["top_drinks_df"]
revenue_trend_df = dashboard_artifacts["revenue_trend_df"]
category_mix_df = dashboard_artifacts["category_mix_df"]
dashboard_metadata = dashboard_artifacts["dashboard_metadata"]

business_dashboard_fig = plot_business_dashboard(
    kpi_df=kpi_df,
    top_drinks_df=top_drinks_df,
    revenue_trend_df=revenue_trend_df,
    category_mix_df=category_mix_df,
    dashboard_metadata=dashboard_metadata,
    source_name=dashboard_source_name,
)

display(business_dashboard_fig)
plt.close(business_dashboard_fig)

display(kpi_df)
display(top_drinks_df.head(10))
display(category_mix_df)

print(
    "Created reusable symbols: dashboard_df, kpi_df, top_drinks_df, revenue_trend_df, category_mix_df, dashboard_metadata, business_dashboard_fig"
)
```

**Execution**

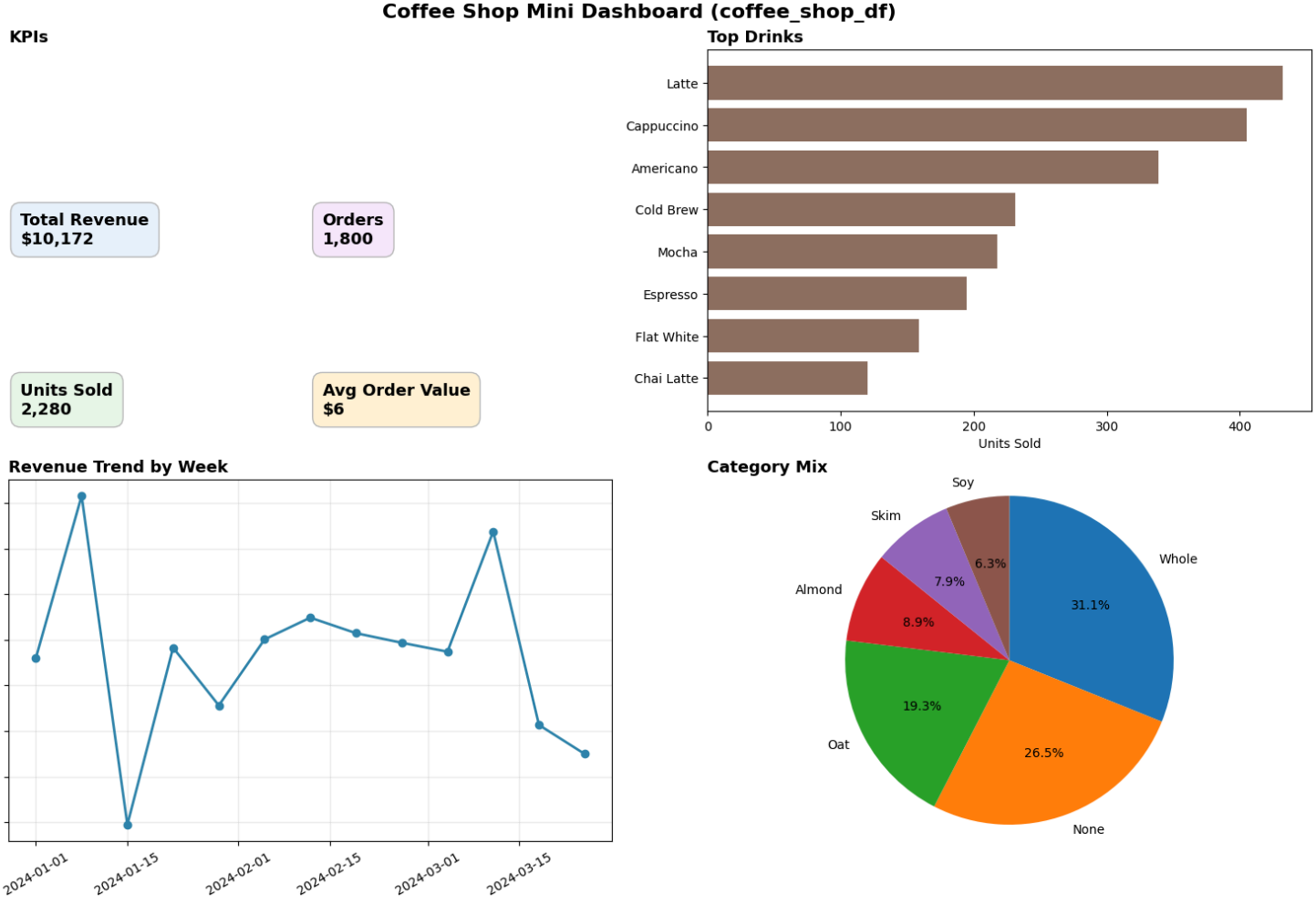
auto-execute via gpt-5.4

**Selected context**

- `generate_coffee_shop_data(n_orders=1500, seed=42)` exists to create a synthetic coffee shop order dataset.
- Prior generated code includes a notebook flow for revealing best-selling drinks with charts and another for time-of-day drink summaries.
- Available helpers include `summarize_drinks_by_time_of_day(df, top_n=5)` and `infer_column(df, candidates, allow_object_fallback=False)` for column detection.
- There is also `build_recommendations_table(promote_df, discontinue_df, bundle_recommendations_df, product_col)` for assembling business action tables.

**Safety notes**

• Looks like an in-memory inspection or plotting task.




| | metric | raw_value | display_value |
|---|---|---|---|
| 0 | Total Revenue | 10171.930000 | $10,172 |
| 1 | Orders | 1800.000000 | 1,800 |
| 2 | Units Sold | 2280.000000 | 2,280 |
| 3 | Avg Order Value | 5.651072 | $6 |

| | product | units_sold |
|---|---|---|
| 0 | Latte | 432 |
| 1 | Cappuccino | 405 |
| 2 | Americano | 339 |
| 3 | Cold Brew | 231 |
| 4 | Mocha | 218 |
| 5 | Espresso | 195 |
| 6 | Flat White | 159 |
| 7 | Chai Latte | 120 |
| 8 | Matcha Latte | 102 |
| 9 | Hot Chocolate | 79 |

| | category | revenue |
|---|---|---|
| 0 | Whole | 3162.21 |
| 1 | None | 2694.20 |
| 2 | Oat | 1968.18 |
| 3 | Almond | 902.89 |
| 4 | Skim | 806.61 |
| 5 | Soy | 637.84 |

```
Created reusable symbols: dashboard_df, kpi_df, top_drinks_df, revenue_trend_df, category_mix_df, dashboard_metadata, business_dashboard_fig
```

In [ ]: